Latex

学习笔记

作者：张斌

组织：河北农业大学　理工系

时间：August 25, 2023

内容收集于网络

目录

1 入门 1

1.1 配置 1

1.2 文档类 1

1.3 颜色 2

2 文字 7

2.1 字体属性 7

2.2 字号 8

2.3 特殊符号 8

3 样式 10

3.1 段落 10

3.2 调整水平间距 11

4 表格 13

4.1 对齐 13

4.2 tabularx 宏包 13

4.3 三线表 15

4.4 问题与解决方法 15

5 图片 16

6 绘图 17

6.1 Tikz 基础 18

6.2 基本图形对象 20

6.3 图形样式 26

6.4 图形变换 28

6.5 示意图 32

6.6 流程图 35

6.7 节点文字位置 37

6.8 常见需求 37

7 公式 42

7.1 公式对齐 42

8 代码 43

9 幻灯片 44

10 自定义命令 45

11 参考文献 46

12 试卷排版 47

13 Linux 篇 48

13.1 WSL 48

第 1 章 入门

1.1 配置 1
1.1.1 外部 PDF 阅读器的设置 1
1.1.2 Sumatra Pdf 反向搜索 1
1.2 文档类 1
1.3 颜色 2

1.1 配置

1.1.1 外部 PDF 阅读器的设置

在 VS Code 里，添加如下配置：

```
 "latex-workshop.view.pdf.viewer":"external",
 "latex-workshop.view.pdf.ref.viewer":"external",
 "latex-workshop.view.pdf.external.viewer.command": "C:/Users/bz/AppData/Local/SumatraPDF/
     SumatraPDF.exe",
 "latex-workshop.view.pdf.external.viewer.args": [
     "%PDF%"
 ],
 "latex-workshop.view.pdf.external.synctex.command": "C:/Users/bz/AppData/Local/SumatraPDF/
     SumatraPDF.exe",
 "latex-workshop.view.pdf.external.synctex.args": [
   "-forward-search",
   "%TEX%",
   "%LINE%",
   "%PDF%",
],
```

1.1.2 Sumatra Pdf 反向搜索

Sumatra 高级设置里添加

```
InverseSearchCmdLine = "C:\Program Files\Microsoft VS Code\Code.exe" "C:\Program Files\Microsoft
     VS Code\resources\app\out\cli.js" --ms-enable-electron-run-as-node -r -g "%f:%l"
EnableTeXEnhancements = true
```

1.2 文档类

- article，文章格式，常用于科技论文、报告、说明文档等。
- proc，基于 article 文档类的一个简单的学术文档模版。
- report，长篇报告，具有章节结构，用于综述、长篇论文、简单书籍等。

- book，书籍，包含章节结构、前言、正文、后记等。
- beamer，幻灯片
- minimal，精简文档，只设定了纸张大小和基本字号。
- standalone，得到大小合适的单页 PDF，用于导出 tikz 绘图或公式。

1.3 颜色

xcolor 基本可用色彩：

black	darkgray	gray	lightgray
yellow	orange	brown	olive
teal	green	lime	pink
red	purple	magenta	violet
blue	cyan	white	

扩充颜色表：`\usepackage[dvipsnames]{xcolor}`

GreenYellow	RubineRed	RoyalPurple	Emerald
Yellow	WildStrawberry	BlueViolet	JungleGreen
Goldenrod	Salmon	Periwinkle	SeaGreen
Dandelion	CarnationPink	CadetBlue	Green
Apricot	Magenta	CornflowerBlue	ForestGreen
Peach	VioletRed	MidnightBlue	PineGreen
Melon	Rhodamine	NavyBlue	LimeGreen
YellowOrange	Mulberry	RoyalBlue	YellowGreen
Orange	RedViolet	Blue	SpringGreen
BurntOrange	Fuchsia	Cerulean	OliveGreen
Bittersweet	Lavender	Cyan	RawSienna
RedOrange	Thistle	ProcessBlue	Sepia
Mahogany	Orchid	SkyBlue	Brown
Maroon	DarkOrchid	Turquoise	Tan
BrickRed	Purple	TealBlue	Gray
Red	Plum	Aquamarine	Black
OrangeRed	Violet	BlueGreen	White

扩充颜色表：`\usepackage[x11names]{xcolor}`

AntiqueWhite1	AntiqueWhite2	AntiqueWhite3	AntiqueWhite4
Aquamarine1	Aquamarine2	Aquamarine3	Aquamarine4
Azure1	Azure2	Azure3	Azure4
Bisque1	Bisque2	Bisque3	Bisque4
Blue1	Blue2	Blue3	Blue4
Brown1	Brown2	Brown3	Brown4
Burlywood1	Burlywood2	Burlywood3	Burlywood4
CadetBlue1	CadetBlue2	CadetBlue3	CadetBlue4
Chartreuse1	Chartreuse2	Chartreuse3	Chartreuse4
Chocolate1	Chocolate2	Chocolate3	Chocolate4
Coral1	Coral2	Coral3	Coral4
Cornsilk1	Cornsilk2	Cornsilk3	Cornsilk4
Cyan1	Cyan2	Cyan3	Cyan4
DarkGoldenrod1	DarkGoldenrod2	DarkGoldenrod3	DarkGoldenrod4
DarkOliveGreen1	DarkOliveGreen2	DarkOliveGreen3	DarkOliveGreen4
DarkOrange1	DarkOrange2	DarkOrange3	DarkOrange4
DarkSeaGreen1	DarkSeaGreen2	DarkSeaGreen3	DarkSeaGreen4
DarkSlateGray1	DarkSlateGray2	DarkSlateGray3	DarkSlateGray4
DeepPink1	DeepPink2	DeepPink3	DeepPink4
DeepSkyBlue1	DeepSkyBlue2	DeepSkyBlue3	DeepSkyBlue4
DodgerBlue1	DodgerBlue2	DodgerBlue3	DodgerBlue4
Firebrick1	Firebrick2	Firebrick3	Firebrick4
Gold1	Gold2	Gold3	Gold4
Goldenrod1	Goldenrod2	Goldenrod3	Goldenrod4
Green1	Green2	Green3	Green4
Honeydew1	Honeydew2	Honeydew3	Honeydew4
HotPink1	HotPink2	HotPink3	HotPink4
IndianRed1	IndianRed2	IndianRed3	IndianRed4
Ivory1	Ivory2	Ivory3	Ivory4
Khaki1	Khaki2	Khaki3	Khaki4
LavenderBlush1	LavenderBlush2	LavenderBlush3	LavenderBlush4
LemonChiffon1	LemonChiffon2	LemonChiffon3	LemonChiffon4
LightBlue1	LightBlue2	LightBlue3	LightBlue4

LightCyan1	LightCyan2	LightCyan3	LightCyan4
LightGoldenrod1	LightGoldenrod2	LightGoldenrod3	LightGoldenrod4
LightPink1	LightPink2	LightPink3	LightPink4
LightSalmon1	LightSalmon2	LightSalmon3	LightSalmon4
LightSkyBlue1	LightSkyBlue2	LightSkyBlue3	LightSkyBlue4
LightSteelBlue1	LightSteelBlue2	LightSteelBlue3	LightSteelBlue4
LightYellow1	LightYellow2	LightYellow3	LightYellow4
Magenta1	Magenta2	Magenta3	Magenta4
Maroon1	Maroon2	Maroon3	Maroon4
MediumOrchid1	MediumOrchid2	MediumOrchid3	MediumOrchid4
MediumPurple1	MediumPurple2	MediumPurple3	MediumPurple4
MistyRose1	MistyRose2	MistyRose3	MistyRose4
NavajoWhite1	NavajoWhite2	NavajoWhite3	NavajoWhite4
OliveDrab1	OliveDrab2	OliveDrab3	OliveDrab4
Orange1	Orange2	Orange3	Orange4
OrangeRed1	OrangeRed2	OrangeRed3	OrangeRed4
Orchid1	Orchid2	Orchid3	Orchid4
PaleGreen1	PaleGreen2	PaleGreen3	PaleGreen4
PaleTurquoise1	PaleTurquoise2	PaleTurquoise3	PaleTurquoise4
PaleVioletRed1	PaleVioletRed2	PaleVioletRed3	PaleVioletRed4
PeachPuff1	PeachPuff2	PeachPuff3	PeachPuff4
Pink1	Pink2	Pink3	Pink4
Plum1	Plum2	Plum3	Plum4
Purple1	Purple2	Purple3	Purple4
Red1	Red2	Red3	Red4
RosyBrown1	RosyBrown2	RosyBrown3	RosyBrown4
RoyalBlue1	RoyalBlue2	RoyalBlue3	RoyalBlue4
Salmon1	Salmon2	Salmon3	Salmon4
SeaGreen1	SeaGreen2	SeaGreen3	SeaGreen4
Seashell1	Seashell2	Seashell3	Seashell4
Sienna1	Sienna2	Sienna3	Sienna4
SkyBlue1	SkyBlue2	SkyBlue3	SkyBlue4
SlateBlue1	SlateBlue2	SlateBlue3	SlateBlue4

SlateGray1	SlateGray2	SlateGray3	SlateGray4
Snow1	Snow2	Snow3	Snow4
SpringGreen1	SpringGreen2	SpringGreen3	SpringGreen4
SteelBlue1	SteelBlue2	SteelBlue3	SteelBlue4
Tan1	Tan2	Tan3	Tan4
Thistle1	Thistle2	Thistle3	Thistle4
Tomato1	Tomato2	Tomato3	Tomato4
Turquoise1	Turquoise2	Turquoise3	Turquoise4
VioletRed1	VioletRed2	VioletRed3	VioletRed4
Wheat1	Wheat2	Wheat3	Wheat4
Yellow1	Yellow2	Yellow3	Yellow4

第 2 章 文字

2.1 字体属性 7
2.1.1 Family 7
2.1.2 Shape 7
2.1.3 Series 8
2.2 字号 8
2.2.1 字号命令 8
2.2.2 中文字号 8
2.3 特殊符号 8
2.3.1 保留字符 8
2.3.2 波浪线 9
2.3.3 角度和摄氏度 9
2.3.4 输出命令 9

2.1 字体属性

2.1.1 Family

有成千上万种字体, 但是基本可以分为三个家族 (family), 即

family	TeX 等价
有衬线字体 (serif)	\rmfamily
无衬线字体 (sans-serif)	\sffamily
等宽字体 (monospaced)	\ttfamily

2.1.2 Shape

每个字体都支持 Shape 属性和 series 属性 (但是不是全支持, 例如 xeCJK 就是通过 AutoFakeSlant 与 AutoFakeBold 来分别解决确实属性的)。Shape 一般有三种：

Shape	TeX 等价
正体 (upright)	\upshape
斜体 (italic)	\itshape
倾斜体 (slanted)	\slshape
小型大写 (Small Capitals)	\scshape

其中`\scshape`理论上对中文无效, 因为中文没有大小写！英文可以用`\lowercase` 或 `\uppercase` 将其转化为小写/大写。

2.1.3 Series

字体的 Series 属性其实是定义了字体的粗细程度, 它一般分为如下三种：

Series	TeX 等价
粗体 (bold)	\bfseries
正常 (middle)	\mdseries
细体 (light)	\lfseries

2.2 字号

2.2.1 字号命令

TeX 里面设置字体大小的开关从小到大如下：

- `\tiny`
- `\scriptsize`
- `\footnotesize`
- `\small`
- `\normalsize`
- `\large`
- `\Large`
- `\Large`
- `\huge`
- `\Huge`

2.2.2 中文字号

使用`\zihao{}`调整字号大小时，西文字号会始终和中文字号保持一致。

0 初号	2 二号	4 四号	6 六号
-0 小初	-2 小二	-4 小四	-6 小六
1 一号	3 三号	5 五号	7 七号
-1 小一	-3 小三	-5 小五	8 八号

2.3 特殊符号

2.3.1 保留字符

LaTeX 中的命令通常是由一个反斜杠加上命令名称，再加上花括号内的参数构成的。

LaTeX 中有许多字符都有特殊的意义，LaTeX 中的保留字符有 `#`,`$`,`%`,`&`,`_`,`{ }`, `\`，这些在正文中都不能直接呈现。

反斜杠用`\textbackslash`表示，其他的符号在前面加 \ 便可以表示了。

2.3.2 波浪线

波浪线 ~ 也不能在 LaTeX 中直接输出，因为它的作用是禁止在该处断行，可以尝试：

```
a $\sim$ b
a \~ b
a \~{} b
a \textasciitilde b
```

a ∼ b　　a b̃　　a ~ b　　a ~b

其中\sim为数学公式中的命令故需要加 $$ 不然会报错，命令\~会自动读取后面的一个字符作为参数，故第二个例子在 b 上加波浪线，而\~{}命令设置参数为空。

2.3.3 角度和摄氏度

角度符号或温度符号（摄氏度）需要借助数学模式。

角度可以使用$^{\circ}$，摄氏度可以使用$^{\circ}\mathrm{C}$。

还可以添加定义

```
\def\degree{${}^{\circ}$}
```

这样可以直接输入 45\degree即可。

另外，使用\usepackage{gensymb}可以直接使用\degree。

2.3.4 输出命令

verbatim环境可原样输出其中的文本，忽视 TeX 命令。

```
\begin{verbatim}
\usepackage{times}
\usepackage{latexsym}
\end{verbatim}
```

输出如下：

```
\usepackage{times}
\usepackage{latexsym}
```

对简短的抄录，可使用\verb| 文字 | 和\verb*| 文字 |。

第 3 章 样式

3.1 段落 10
3.1.1 分段 10
3.1.2 缩进 10
3.1.3 行距 10
3.1.4 Latex 与 Word 行距的对应关系 11
3.1.5 段间距 11
3.2 调整水平间距 11
3.2.1 不可换行的水平间距 11
3.2.2 可换行的水平间距 12

3.1 段落

3.1.1 分段

- 最长用的分段方式是空行，一个空行或多个空行都可以
- 使用`\par`命令
- \\手动换行，不是分段。区别是分段会自动缩进，而手动换行不会自动缩进。

3.1.2 缩进

缩进由`\parindent`控制。

中文文档类 (如 ctexart) 自动设置段落缩进为两个汉字宽。

如果要在某一段开头临时禁用缩进，可在段前使用`\noindent`命令。

如果要在本来没有缩进的地方使用缩进，可以用`\indent`命令产生一个长为`\parindent`的缩进。

3.1.3 行距

行距就是相邻两行文字之间的距离。在下面的图片中，两个下划线之间的长度就叫行距。

行距的调节一般使用倍数，比如两倍行距。而“单倍行距”又根据字体、字号、软件的不同而改变（不同软件中有不同的定义，没有一个通用的值）。在 LaTeX 中，默认的单倍行距是字体字号的 1.2 倍。

如果要修改全局的默认行距，则在导言区加入“`\linespread`”命令进行修改即可。比如想要修改成 2 倍行距，用如下代码修改即可：

```
\linespread{2}
```

如果只想对一小部分做更改，可以用大括号把这一段括起来，在调用\linespread命令后，还要添加\selectfont命令刷新行距信息。

```
{
\linespread{2.0} \selectfont
% 两倍行距的文字
}
```

\linespread命令不仅可以改变正文行距，同时也把目录、脚注、图表标题的行距也改了。使用setspace 宏包仅改变正文的行距，包括调整数学公式、浮动体的行距。

```
\usepackage{setspace}
\singlespacing % 1倍字体高度
\onehalfspacing % 1.5倍字体高度
\doublespacing % 2倍字体高度
\setstretch{1.25} % 任意行距
```

同时也可以在正文中进行局部环境设置

```
\begin{spacing}{2.0}
  % 这里是两倍行距
\end{spacing}
```

3.1.4 Latex 与 Word 行距的对应关系

同样是一倍行距，LaTeX 看起来就是要比 Word 要紧密一些，原因就在于前面所说的 LaTeX 中，默认的行距是字体字号的 1.2 倍，而新版本的 Word 默认行距是字体字号的 1.3 倍。那怎么将 LaTeX 的行距设置得跟 Word 一样呢？很简单，就是“乘以 1.3 除以 1.2”，比如在 Word 中要求的是 1.5 倍行距，则在 LaTeX 中设置行距为：

$$1.5 \times 1.3 \div 1.2 = 1.625$$

3.1.5 段间距

段与段之间的垂直距离，由变量\parskip控制。

\parskip的默认值是橡皮长度 0pt plus 1pt。

在中文排版中，常使用

```
\setlength\parskip{0pt}
```

把段间距定义为固定长度。

3.2 调整水平间距

3.2.1 不可换行的水平间距

- \thinspace 或 \,　　0.1667 em

- \negthinspace -0.1667 em
- \enspace 0.5 em
- \nobreakspace 或 ~ 空格

3.2.2 可换行的水平间距

- \quad 1 em
- \qquad 2 em
- \enskip 0.5 em
- \␣ 空格
- \hspace{距离} 产生指定的水平间距
- \hfill 橡皮长度，\hspace{\fill}的缩写，把内容均匀分布在一行
- \hspace{\stretch{2}} 可设定填充的比例
- \hrulefill 用横线填充
- \dotfill 用圆点填充
- 幻影盒子，产生与参数内容一样大小的盒子

第 4 章 表格

4.1 对齐 13
4.1.1 表格整体居中 13
4.2 tabularx 宏包 13
4.2.1 表格内自动换行 13
4.2.2 不同宽度的 X 列 14
4.2.3 让 tabularx 表格中的内容垂直居中 15
4.3 三线表 15
4.3.1 三线表竖线断掉 15
4.4 问题与解决方法 15
4.4.1 解决表格过宽问题，自适应调整宽度 15

4.1 对齐

4.1.1 表格整体居中

为什么使用了\begin{center} - \end{center}但表格还是没有整体居中？

- 它只是居中了\begin{table} - \end{table}；
- 如果里面嵌套了\begin{tabular} - \end{tabular}，那么表格还是不会居中。

要使得表格居中，则需要在\begin{table}后面附加\centering，才能使得表格整体居中。

```
\begin{center}
   \begin{table}[h]
      \centering
      \begin{tabular}

      \end{tabular}
   \end{table}
\end{center}
```

4.2 tabularx 宏包

4.2.1 表格内自动换行

使用 tabularx 宏包来帮助我们实现表格内自动换行与自动计算列宽。

```
\usepackage{tabularx}
```

语法说明：

```
\begin{tabularx}{widthi}[posi]{preamblei}
\end{tabularx}
```

例: \begin{tabularx}{10.5cm}{|X|X|X|} % 表格总宽度为 10.5cm，共 3 列，宽度均相同。每列宽度为 10.5/3=3.5。是自动计算出来的。如将上表设置改为\begin{tabularx}{\linewidth}{|p3cm|X|X|}，则表格的总宽度是行宽，第 1 列列宽为 3cm，其他两列的列宽自动计算。

tabularx 定义了一个新的字母—大写的 X。使用 X 指定的列，将会自动换行并向左对齐。那么，向右对齐呢？居中对齐呢？

```
\begin{tabularx}{0.8\textwidth} {
   | >{\raggedright\arraybackslash}X
   | >{\centering\arraybackslash}X
   | >{\raggedleft\arraybackslash}X
   | }
   \hline
   abc & edf & ghi\\
   \hline
   edf & abc & ghi\\
   \hline
 \end{tabularx}
```

\raggedright和\centering分别对应了向左对齐（参差不齐的右边即左对齐）、居中对齐。但在 tabularx 中不能直接使用，他们会破坏换行符\\。为此，tabularx 定义了 \arraybackslash 来解决这个问题。使用方法就是直接加在后面，比如 >{\raggedright\arraybackslash}X 就是进行左对齐，>和{}不能少！

每一个 tabularx 表格至少要有一个 X 列！虽然兼容 l,c,r 写法，但纯 l,c,r 组成的 tabularx 表格会出错的！

abc	edf	ghi
edf	abc	ghi

4.2.2 不同宽度的 X 列

通常，单个表中的所有 X 列都设置为相同的宽度，但是可以让 tabularx 将它们设置为不同的宽度。下面是文档中的例子。

```
{
   >{\hsize=.5\hsize\linewidth=\hsize}X
   >{\hsize=1.5\hsize\linewidth=\hsize}X
}
```

指定两列，第二列的宽度是第一列的三倍。但是，如果你想进行这种神奇地操作，则应遵循以下两条规则：

- 确保所有 X 列的宽度之和保持不变。（在上面的示例中，新宽度加起来仍然是默认宽度的两倍，即等于两个标准宽度相同 X 列）
- 不要使用跨越任何 X 列的 \multicolumn

示例：三列宽度 1:2:3

abc	edf	ghi
edf	abc	ghi

4.2.3 让 tabularx 表格中的内容垂直居中

要让 tabularx 表格中的内容上下垂直居中，可以使用 m{width} 形式的列类型。该列类型会将内容垂直居中，并指定列宽为width。具体实现方式如下：

```
\newcolumntype{Y}{>{\centering\arraybackslash}m{0.3\textwidth}} %定义新的列类型Y，宽度为页面的 30%
\newcolumntype{Z}{>{\raggedright\arraybackslash}X} %定义新的列类型Z，使用X列类型但是左对齐
```

4.3 三线表

4.3.1 三线表竖线断掉

H	e	l	l	o
01001000	01100101	01101100	01101100	01101111

解决方法：把\toprule和\bottomrule互换，中间用\hline。

```
\begin{tabular}{|c|c|c|c|c|}
   \bottomrule
  H & e & l & l & o \\ \hline
  01001000 & 01100101 & 01101100 & 01101100 & 01101111 \\ \toprule
\end{tabular}
```

H	e	l	l	o
01001000	01100101	01101100	01101100	01101111

4.4 问题与解决方法

4.4.1 解决表格过宽问题，自适应调整宽度

```
\usepackage{graphicx} # 记得加宏包
\resizebox{\linewidth}{!}{ #此处! 表示根据根据宽高比进行自适应缩放
\begin{tabular}...
....
....
\end{tabular}
} # 注意加的位置在\begin{tabular}和\end{tabular}前后
```

第 5 章 图片

第 6 章 绘图

6.1 Tikz 基础 18
6.1.1 TikZ 是什么？ 18
6.1.2 TikZ 使用 18
6.1.3 坐标系统 19
6.1.4 常用元素 19
6.2 基本图形对象 20
6.2.1 直线和矩形 20
6.2.2 圆、椭圆、弧 22
6.2.3 曲线 23
6.2.4 箭头 24
6.2.5 网格和坐标轴 24
6.2.6 函数 25
6.2.7 显示绘图区域边界 26
6.3 图形样式 26
6.3.1 线型和线宽 26
6.3.2 颜色和填充 27
6.3.3 自定义样式 28
6.4 图形变换 28
6.4.1 平移 (shift) 28
6.4.2 缩放 (scale) 29
6.4.3 旋转 (rotate) 30
6.4.4 倾斜 (slant) 31
6.5 示意图 32
6.5.1 文字节点 32
6.5.2 图形文字节点 33
6.5.3 为绘制的图形添加文字节点 33
6.5.4 绘制一颗树 34
6.5.5 绘制函数图像 35
6.6 流程图 35
6.7 节点文字位置 37
6.7.1 文字换行 37
6.7.2 折线文字位置 37
6.8 常见需求 37
6.8.1 描点 37
6.8.2 指定标签位置 38
6.8.3 为线添加标签 38
6.8.4 设定数值包含根式的元素 38

6.8.5 标注直角符号 39
6.8.6 foreach 循环功能 39
6.8.7 利用循环功能绘制时间线 39
6.8.8 多变量同时循环 40
6.8.9 嵌套循环 40
6.8.10 长图片跨页 40

6.1 Tikz 基础

6.1.1 TikZ 是什么？

PGF: Portable Graphics Format
由 beamer 的作者 Till Tantau 开发
PGF 可以精确的绘制复杂的几何图形及各种曲线
TikZ 是 PGF 的前端，使用时只需调用 tikz 宏包
tikz 图是矢量图。

- 矢量图
  - 由数学曲线计算得到，保留图的几何特征，不因缩放而影响清晰度；
  - 具有很高的精度，且通常文件很小；
  - 用来表达一些具有逻辑的示意图，如流程图、数学曲线等。
- 点阵图
  - 以矩阵形式存储每个像素的色值，缩放和坐标变换会影响清晰度；
  - 图像采样得到，可以描述极其复杂的图形；
  - 通常文件较大。

6.1.2 TikZ 使用

```
% 使用方法:
\usepackage{tikz}
% 根据需要调用tikz 扩展
\usetikzlibrary{arrows,backgrounds,...}
% 使用tikz命令
\tikz\draw(0,0) -- (3,1);
\tikz{\draw(0,0) -- (3,1);}
% 使用tikzpicture环境
\begin{tikzpicture}
   \draw(0,0) -- (3,1);
\end{tikzpicture}
```

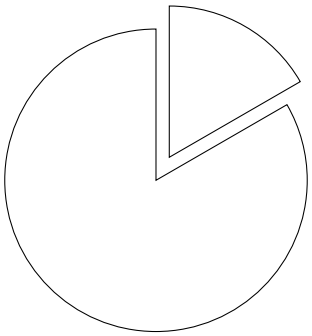
- tikz 基于坐标绘图，原点在左下角
- 每条 tikz 绘图命令以分号结束
- 使用长度或坐标时数字可以带单位（cm, em, pt...）也可不带

pt　1pt = 1/72.27 英寸 =0.035146 厘米 (cm)
bp　1bp = 1/72 英寸
mm　毫米
cm　厘米
in　英寸
ex　当前字体中“x”的高度
em　当前字体的一个“quad”的宽度（在以前，它是字母“M”的宽度，现在不是了）

6.1.3 坐标系统

- 使用直角坐标 (x,y)，如 (0,1)，(0.4cm,5pt)
- 使用极坐标 (θ:r)，如 (30,1cm)
- 使用相对位置：
  - 一个加号：+(0,5pt) 从指定的点向上移 5pt
  - 两个加号：++(0,5pt) 从指定的点向上移 5pt，并把此点作为新的当前点

```
\begin{tikzpicture}[scale=2]
   \draw(0,0)--(90:1cm)arc(90:360:1cm)
   arc(0:30:1cm)--cycle;
   \draw(60:5pt)-- +(30:1cm) arc (30:90:1cm) -- cycle;
\end{tikzpicture}
```



6.1.4 常用元素

几何中常见的元素包括: 点、线、圆、面。在 TikZ 下分别通过以下关键词定义:

元素	关键词	示例	效果
点	`\coordinate`	`\coordinate (A) at (1,-1);` `\coordinate (B) at (45:2);`	创建点 A，位于 (1,-1) 创建点 B，位于以圆点为圆心，2 为半径，逆时针旋转 45° 处
线	`--`	`\draw [dashed] (A)--(B)`	用虚线连接 A 与 B
圆	`circle`	`\draw (1,2) circle (1cm)`	以 (1,2) 为圆心，1cm 为半径画圆
面	`\fill`	`\fill[gray!50](A)--(B)--(C)`	以 50% 透明度灰色填充三角形 ABC

6.2 基本图形对象

6.2.1 直线和矩形

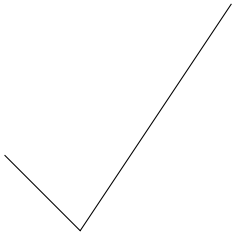
使用 PGF 绘图的基本语法是：

```
\draw[option]...;
```

其中\draw称为绘图命令；后面的... 部分称为操作；而 [ ] 中的内容称为选项，也就是说 [ ] 内可以不写内容。

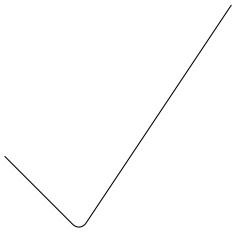
如果我们要绘制一条直线的话，我们只需要在\draw命令后面输入点的坐标并使用--连结起来即可。比如：

```
\begin{tikzpicture}
   \draw (1,3)--(2,2)--(4,5);
\end{tikzpicture}
```



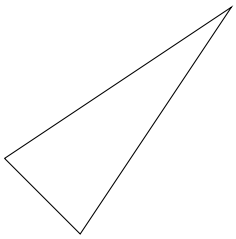
可以通过 [option] 将图片 1 中锋利的角变成圆角，在 option 处填写 rounded corners 即可：

```
\begin{tikzpicture}
   \draw[rounded corners] (1,3)--(2,2)--(4,5);
\end{tikzpicture}
```



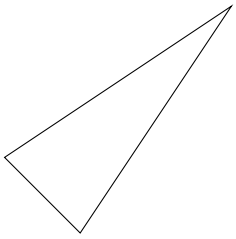
也可以将其首尾相连，使他成为一个“封闭”图形。做法也很简单，就是让最后一个点的坐标与起点相同即可：

```
\begin{tikzpicture}
   \draw (1,3)--(2,2)--(4,5)--(1,3);
\end{tikzpicture}
```

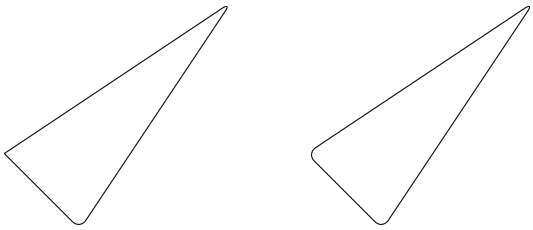


但图片三并非真正意义上的封闭图形，因为那只是我们人为的设置了终点和起点一致而已。如果想要绘制真正意义上的封闭图形，那我们需要使用 cycle 操作:

```
\begin{tikzpicture}
   \draw (1,3)--(2,2)--(4,5)--cycle;
\end{tikzpicture}
```



你会发现两张图片似乎没有什么区别，但是，如果将所有的角都换成圆角，那么区别就立刻出现了：



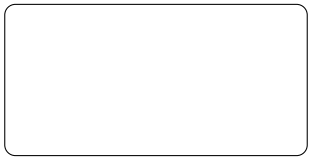
伪封闭的图形它的左上顶点并未被圆角化，而真封闭的图形中所有角均被圆角化了。

下面我们再来看看如何绘制一个矩形。

```
\begin{tikzpicture}
   \draw (0,0) rectangle (4,2);
\end{tikzpicture}
```

可见绘制矩形使用到的操作是 rectangle。我们只需要给出矩形的一对对角顶点然后将 rectangle 写在两者中间即可。比如图中的两个对角顶点分别是 (0,0) 和 (4,2)。矩形也有圆角选项，设置方式与之前一样：

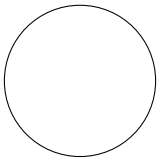
```
\begin{tikzpicture}
   \draw[rounded corners] (0,0) rectangle (4,2);
\end{tikzpicture}
```



6.2.2 圆、椭圆、弧

画一个圆的命令十分的简单，给出圆心坐标和半径，然后将 circle 操作写在两者之间即可，注意半径值需要用小括号括起来：

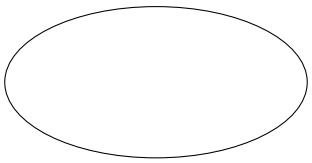
```
\begin{tikzpicture}
   \draw (1,1) circle (1);
\end{tikzpicture}
```



绘制了一个圆心在 (1,1)，半径为 1 的圆。

绘制椭圆的方式也很简单，只要给出椭圆的重心和长轴长、短轴长，然后将 ellipse 操作写在两者中间即可，注意，长轴长和短轴长需要用括号括起来，两者之间用 and 隔开：

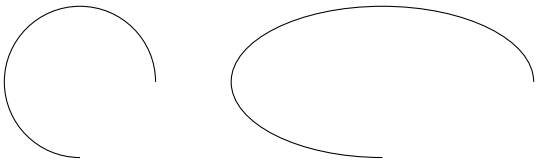
```
\begin{tikzpicture}
   \draw (1,1) ellipse (2 and 1);
\end{tikzpicture}
```



绘制了一个中心在 (1,1)，长轴长为 2、短轴长为 1 的椭圆。

绘制圆弧或者椭圆弧我们只需要使用 arc 操作并给出相应角度即可：

```
\begin{tikzpicture}
   \draw (1 ,1) arc (0:270:1);
   \draw (6 ,1) arc (0:270:2 and 1);
\end{tikzpicture}
```



其中，参数 0:270 代表的是 0° 到 270°，即分别绘制了一个圆心在 (1 ,1)，半径为 1 的四分之三圆弧和一个中心在 (6 ,1)，长轴长为 2、短轴长为 1 的四分之三椭圆弧。在终止角度之后，使用一个冒号隔开终止角度与半径（长轴长 and 短轴长）。

6.2.3 曲线

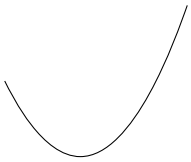
我们把直线的 -- 换成 . . ，就得到贝塞尔曲线，它需要至少一个控制点 (controls)。

```
\begin{tikzpicture}
   \draw (1,3) .. controls (2,2)..(4,5);
   \draw (5,3) .. controls (6,2) and (8,5)..(9,4);
\end{tikzpicture}
```

(5,3) 代表曲线的起点，(6,2)(8,5) 代表曲线的控制点，(9,4) 代表曲线的终点。

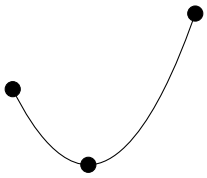
抛物线用 parabola 操作，bend 操作可以指明顶点。

```
\begin{tikzpicture}
   \draw (5,1) parabola bend (6,0) (7.414,2);
\end{tikzpicture}
```



(5,1) 代表抛物线的起点，(6,0) 代表抛物线的顶点，(7.414,2) 代表抛物线的终点。如果我们想要将这些点也进行标记那我们可以使用`\filldraw`命令:

```
\begin{tikzpicture}
   \draw (5,1) parabola bend (6,0) (7.414 ,2);
   \filldraw (5,1) circle (.1)
   (6,0) circle (.1)
   (7.414 ,2) circle (.1);
\end{tikzpicture}
```

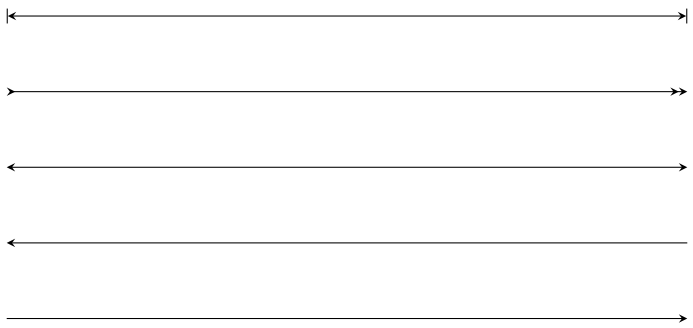


利用`\filldraw`命令在坐标 (5,1)、(6,0)、(7.414 ,2) 填充半径为 0.1pt 的圆。

6.2.4 箭头

一些绘制箭头的命令如下：

```
\begin{tikzpicture}
   \draw [->] (0,0)--(9,0);
   \draw [<-] (0,1)--(9,1);
   \draw [<->] (0,2)--(9,2);
   \draw [>->>] (0,3)--(9,3);
   \draw [|<->|] (0,4)--(9,4);
\end{tikzpicture}
```



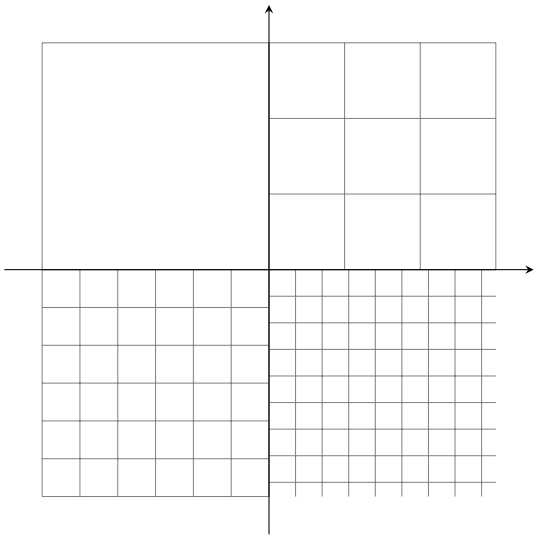
绘制箭头的命令很简单。[ ] 仍是选项，这个选项中是你要绘制的箭头的形式，其中 < 和 > 代表了箭头两侧的指向，而-表示线。因为我们的箭头都是有限长度的，所以，我们需要在 [ ] 后面指定箭头的起点和终点，而线是直线，所以我们仍使用--连接起点和终点。

更改默认箭头样式：

```
\tikzset{>=stealth}
```

6.2.5 网格和坐标轴

```
\begin{tikzpicture}
   \draw [help lines ,step=0.5] (-3,-3) grid (0,0);
   \draw [help lines ,step=3] (-3,3) grid (0,0);
   \draw [help lines] (0,0) grid (3,3);
   \draw [help lines,step=10pt] (0,0) grid (3,-3);
   \draw [->] (-3.5,0) -- (3.5,0);
   \draw [->] (0,-3.5) -- (0,3.5);
\end{tikzpicture}
```



grid 操作表示绘制网格，需要起止点两个参数。help lines 表示辅助线，为 0.2pt 的灰线。step 是步长，即每一个小单元格的边长（小单元格均为正方形），其单位是 cm，缺省步长是 1cm。可设置为其它单位。

6.2.6 函数

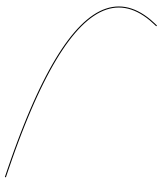
原生 TikZ 作图使用到的命令是

```
\draw [options] plot (\x,{function(\x)});
```

绘制二次函数 $y = -x^2 + 3x$ 图像：

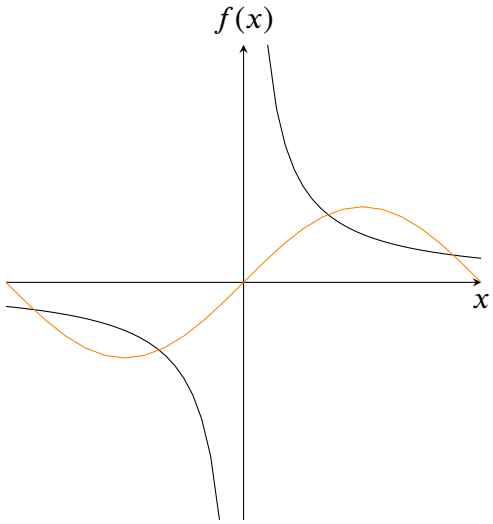
```
\draw [smooth, domain = 0:2] plot (\x,{-\x^2 + 3*\x});
```

smooth：平滑效果适合绘制函数图像；domain：定义域为 [0,2]



使用 π：

```
\draw[->] (-pi,0) -- (pi,0) node[below] {$x$};
\draw[->] (0,-pi) -- (0,pi) node[above] {$f(x)$};
\draw[domain=-pi:-1/pi] plot(\x,{1/\x});
\draw[domain=1/pi:pi] plot(\x,{1/\x});
\draw[orange,domain=-pi:pi] plot(\x,{sin(\x r)});
```

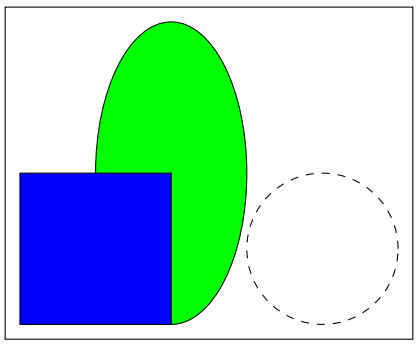


6.2.7 显示绘图区域边界

```
\usetikzlibrary{backgrounds} % 写在导言区
```

例：

```
\begin{tikzpicture}
   [scale=2, show background rectangle]
   \draw[style=dashed] (2,.5) circle (0.5);
   \draw[fill=green] (1,1) ellipse (.5 and 1);
   \draw[fill=blue] (0,0) rectangle (1,1);
\end{tikzpicture}
```



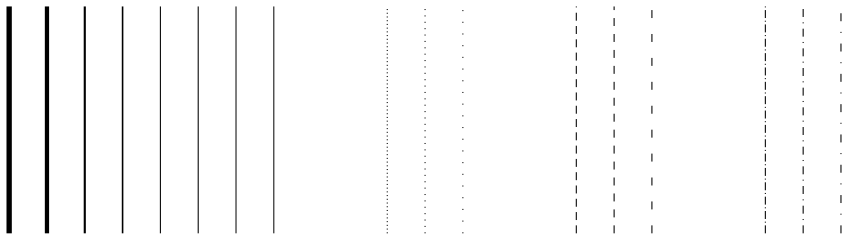
6.3 图形样式

6.3.1 线型和线宽

PGF 中线条的缺省宽度是 0.4pt，线型是实线。

- 直线的宽度参数有 ultra thin(超细)、very thin(非常细)、thin(细)、semithick(半粗)、thick(粗)、very thick(非常粗)、ultra thick(超粗)
- 直线的类型参数有 dashed(虚线)、dotted(虚点)、dash dot(点划线)、solid(实线)
- 虚线的样式有 densely(较密) 和 loosely(较疏)

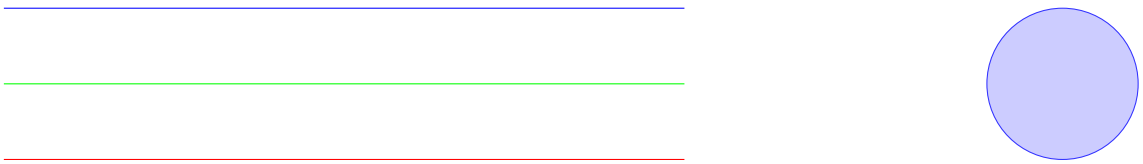
```
\begin{tikzpicture}
   \draw [line width =2pt] (0,0)--(0,3);
   \draw [ultra thick] (0.5,0)--(0.5,3);
   \draw [thick] (1,0)--(1,3);
   \draw [semithick] (1.5,0)--(1.5,3);
   \draw (2,0)--(2,3);
   \draw [thin] (2.5,0)--(2.5,3);
   \draw [very thin] (3,0)--(3,3);
   \draw [ultra thin] (3.5,0)--(3.5,3);
   \draw [densely dotted] (5,0)--(5,3);
   \draw [dotted] (5.5,0)--(5.5,3);
   \draw [loosely dotted] (6,0)--(6,3);
   \draw [densely dashed] (7.5,0)--(7.5,3);
   \draw [dashed] (8,0)--(8,3);
   \draw [loosely dashed] (8.5,0)--(8.5,3);
   \draw [densely dash dot] (10,0)--(10,3);
   \draw [dash dot] (10.5,0)--(10.5,3);
   \draw [loosely dash dot] (11,0)--(11,3);
\end{tikzpicture}
```



6.3.2 颜色和填充

PGF 可以结合 xcolor 宏包的色彩功能。颜色和填充的用法见下例，其中`\filldraw`命令可以用不同颜色画线和填充。注意封闭路径才可以填充！

```
\begin{tikzpicture}
   \draw[red] (0,0) --(9,0);
   \draw[green] (0,1) --(9,1);
   \draw[blue] (0,2) --(9,2);
   \filldraw[draw=blue!80,fill=blue!20] (14 ,1) circle (1);
\end{tikzpicture}
```



上例中如果我们想设置直线的颜色那么我们可以直接在`\draw`命令后面的选项中填写颜色即可。`\filldraw`命令的选项中，draw 是为封闭图形的边界设置颜色，fill 对封闭图形的内部填充颜色。

6.3.3 自定义样式

PGF 可以自定义一些样式，这些样式一旦被定义了就可以像面向对象的类一样被继承。也就是说我们可以在导言区定义一个样式，然后将来我们可以直接在文本区调用它。比如：

```
\tikzset{
   dline/.style ={color = blue, line width =2pt}
   }
\begin{document}
   \begin{tikzpicture}
      \draw[dline] (0,0) --(9,0);
   \end{tikzpicture}
\end{document}
```


除了用`\tikzset`命令定义样式，我们也可以在 tikzpicture 环境头部声明样式。前者是全局有效，后者则是局部范围有效。

```
\begin{tikzpicture}[dline /. style ={line width =2pt}]
   \draw[dline] (0,0) --(9,0);
\end{tikzpicture}
```

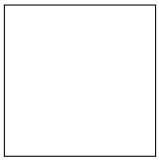
6.4 图形变换

6.4.1 平移 (shift)

1. 整体平移

首先，先来画一个正方形：

```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
\end{tikzpicture}
```

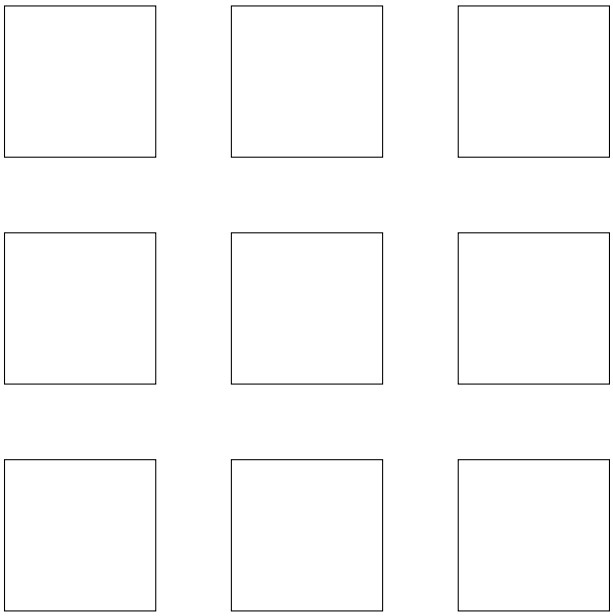


接下来，对其进行平移：

```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
   \draw[shift ={(3 ,0)}] (0,0) rectangle (2,2);
   \draw[shift ={(0 ,3)}] (0,0) rectangle (2,2);
   \draw[shift ={(0 ,-3)}] (0,0) rectangle (2,2);
   \draw[shift ={(-3 ,0)}] (0,0) rectangle (2,2);
   \draw[shift ={(3 ,3)}] (0,0) rectangle (2,2);
```

```
    \draw[shift ={(-3 ,3)}] (0,0) rectangle (2,2);
    \draw[shift ={(3 ,-3)}] (0,0) rectangle (2,2);
    \draw[shift ={(-3 ,-3)}] (0,0) rectangle (2,2);
\end{tikzpicture}
```

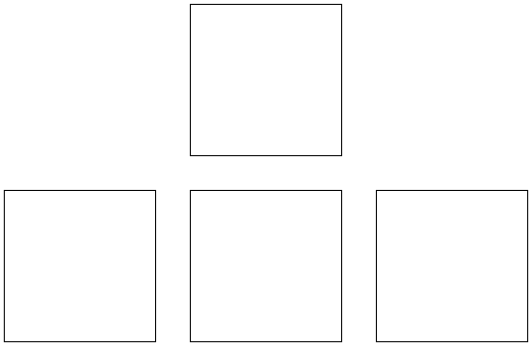
shift 表示整体平移。



2. 水平缩放和垂直缩放

xshift 表示水平平移；yshift 表示垂直平移，后面直接跟平移的距离即可，单位是 pt。需要注意的是如果指定了平移的方向那么后面就不能使用坐标了，而只能自定义平移距离。

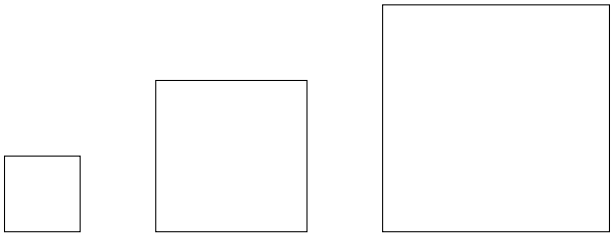
```
\begin{tikzpicture}
    \draw (0,0) rectangle (2,2);
    \draw[xshift =70] (0,0) rectangle (2,2);
    \draw[xshift =-70] (0,0) rectangle (2,2);
    \draw[yshift =70pt] (0,0) rectangle (2,2);
\end{tikzpicture}
```



6.4.2 缩放 (scale)

1. 对图形进行整体缩放

```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
   \draw[shift ={(3,0)},scale =1.5] (0,0) rectangle (2,2);
   \draw[shift ={(-2 ,0)},scale =0.5] (0,0) rectangle (2,2);
\end{tikzpicture}
```

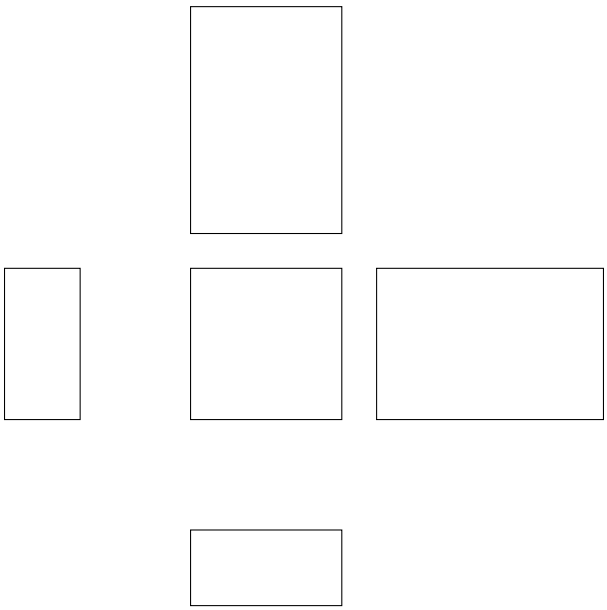


这里在缩放的基础上又进行了平移是为了方便说明所有缩放的效果。

scale $= \alpha$ ：缩放 α 倍。若 $\alpha > 1$ 则代表放大，$\alpha < 1$ 则代表缩小。

2. 水平缩放和垂直缩放

```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
   \draw[xshift =70pt ,xscale =1.5] (0,0) rectangle (2,2);
   \draw[yshift =70pt ,yscale =1.5] (0,0) rectangle (2,2);
   \draw[xshift =-70pt ,xscale =0.5] (0,0) rectangle (2,2);
   \draw[yshift =-70pt ,yscale =0.5] (0,0) rectangle (2,2);
\end{tikzpicture}
```

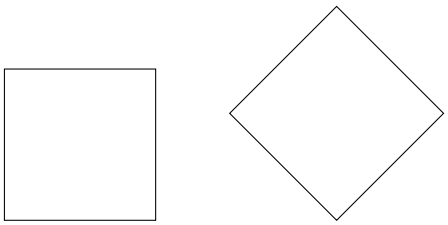


xscale 代表水平方向缩放，yscale 代表垂直方向缩放。

6.4.3 旋转 (rotate)

1. 围绕起点旋转

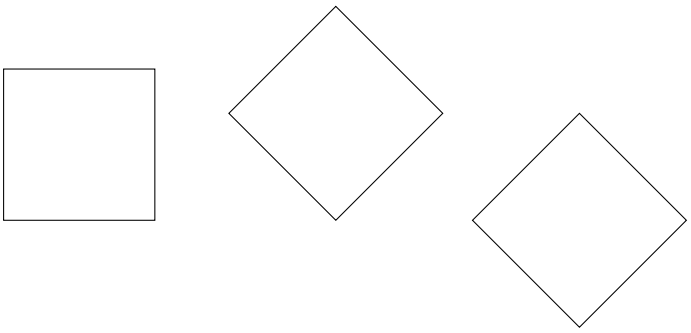
```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
   \draw[xshift =125pt ,rotate =45] (0,0) rectangle (2,2);
\end{tikzpicture}
```



rotate 后面的数值写要旋转的角度，正数表示逆时针旋转相应角度，而负数表示顺时针旋转相应角度，这与数学上的正方向规定是一致的。而且如果没有指定对某个点进行旋转的话那么默认是围绕起点进行旋转。

2. 围绕定点旋转

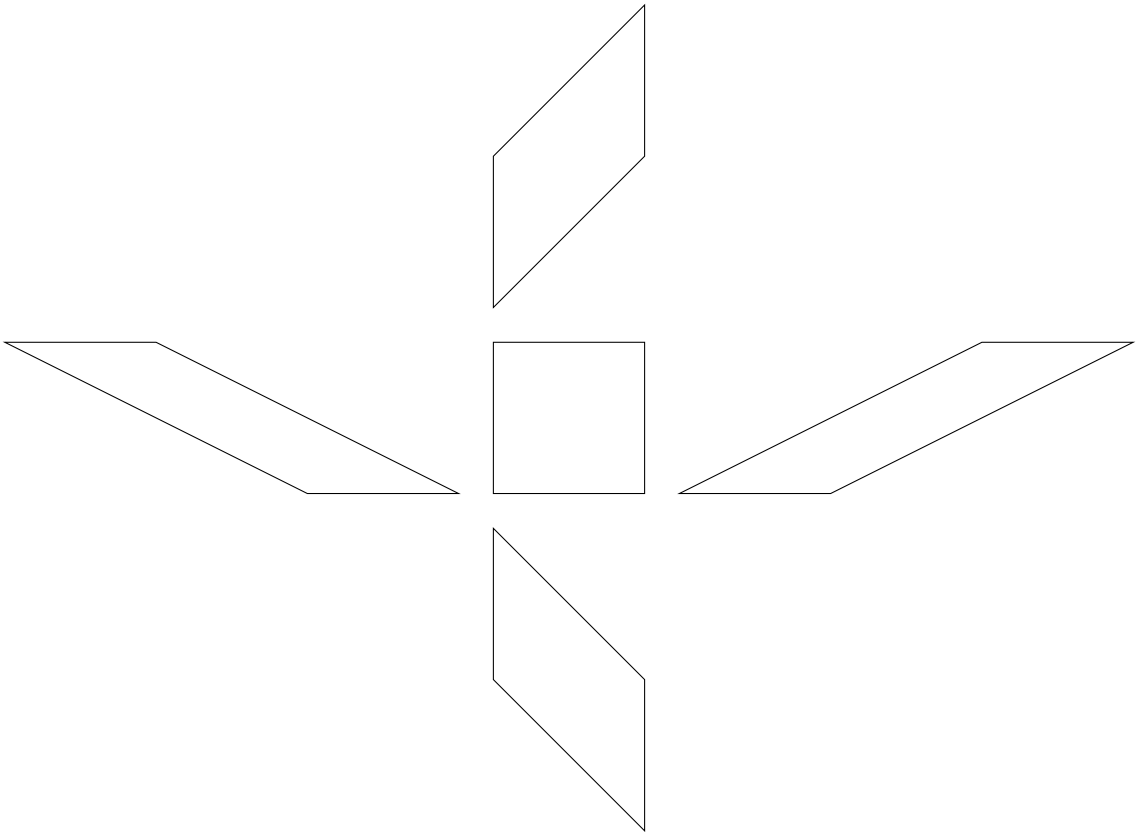
```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
   \draw[xshift =125pt ,rotate =45] (0,0) rectangle (2,2);
   \draw[xshift =200pt ,rotate around ={45:(2 ,0)}] (0,0) rectangle (2,2);
\end{tikzpicture}
```



定点旋转的只需要在 rotate around 后面的中写上旋转的角度和指定点的坐标即可。旋转的方向与非定点旋转时的方向规定一致。

6.4.4 倾斜 (slant)

```
\begin{tikzpicture}
   \draw (0,0) rectangle (2,2);
   \draw[xshift =70pt ,xslant =2] (0,0) rectangle (2,2);
   \draw[xshift =-70pt ,xslant =-2] (0,0) rectangle (2,2);
   \draw[yshift =70pt ,yslant =1] (0,0) rectangle (2,2);
   \draw[yshift =-70pt ,yslant =-1] (0,0) rectangle (2,2);
\end{tikzpicture}
```



xslant 表示水平倾斜，yslant 表示垂直倾斜。其值越大，代表倾斜的角度越大。

6.5 示意图

6.5.1 文字节点

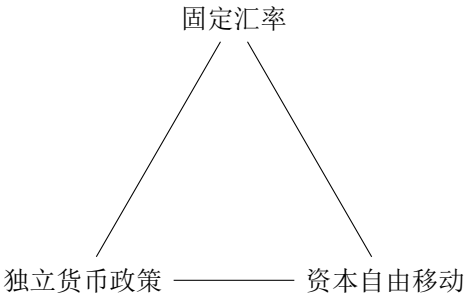
利用 Tikz 宏包绘制文字节点需要调用\node命令，其语法格式如下：

```
\node [<options>] (<name>) at (<coordinate>) {<text>}
```

语法中各参数意义如下：

1. <options>：可选参数，用来设定绘制文字节点的样式或图形
2. <name>：文字节点的名称
3. <coordinate>：用来指定节点的位置
4. <text>：文字节点要显示的文字内容

```
\begin{tikzpicture}
   \node (A) at (0,0) {\footnotesize 独立货币政策};
   \node (B) at (4,0) {\footnotesize 资本自由移动};
   \node (C) at (60:4) {\footnotesize 固定汇率};
   \draw (A) -- (B) -- (C) -- (A);
\end{tikzpicture}
```

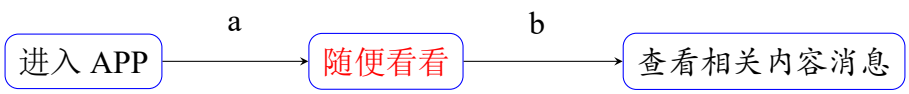


6.5.2 图形文字节点

```
\tikzset{
   box/.style ={
   rectangle, % 矩形节点
   rounded corners =5pt, % 圆角
   minimum width =50pt, % 最小宽度
   minimum height =20pt, % 最小高度
   inner sep=5pt, % 文字和边框的距离
   draw=blue} % 边框颜色
}
```

上述代码设置一个名为 box 的样式，它的边框是矩形，有圆角；它有最小宽度、高度、文字和边框的距离，边框和填充颜色等属性。

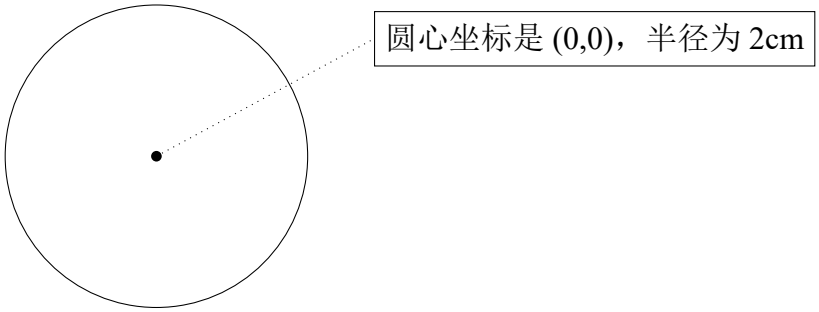
```
\begin{tikzpicture}
   \node[box] (A) at(0,0) {进入APP};
   \node[box, text=red] (B) at(4,0) {随便看看};
   \node[box, node font={\kaishu}] (C) at(9,0) {查看相关内容消息};
   \draw[->] (A)--(B);
   \draw[->] (B)--(C);
   \node at(2,0.5) {a};
   \node at(6,0.5) {b};
\end{tikzpicture}
```



上述代码中，文字节点样式为 [box]，`text=red`表示矩形内文字的颜色为红色；`node font = {\kaishu}`表示矩形内文字为楷体。

6.5.3 为绘制的图形添加文字节点

```
\begin{tikzpicture}
   \draw (0,0) circle [radius=2];
   \fill (0,0) circle [radius=2pt];
   \node[draw] (P) at (15:6) {圆心坐标是(0,0)，半径为2cm};
   \draw[dotted] (0,0) -- (P.west);
\end{tikzpicture}
```



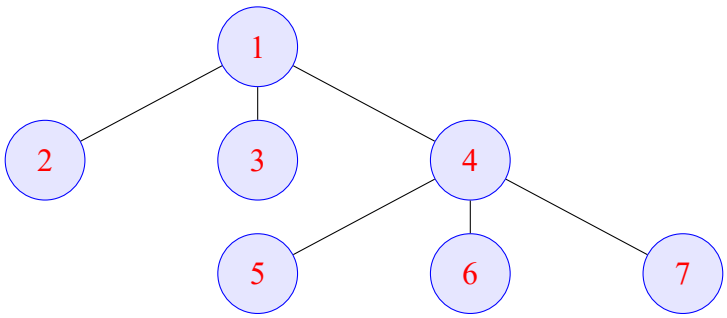
\draw命令中可以使用某个节点的相对位置，以“东南西北”的方式命名。P.west 表示 P 节点的左边；east 表示右边；north 表示上边；south 表示下边。

6.5.4 绘制一颗树

```
\tikzset{
   cir/.style ={
   circle, % 圆形节点
   minimum width =30pt, % 最小宽度
   minimum height =30pt, % 最小高度
   fill =blue!10, % 填充色
   text =red, % 文字红色
   inner sep=5pt, % 文字和边框的距离
   draw=blue} % 边框颜色
}
```

上述代码设置一个名为 cir 的样式，它是圆形节点，它有最小宽度、高度、文字和边框的距离，边框和填充颜色等属性。

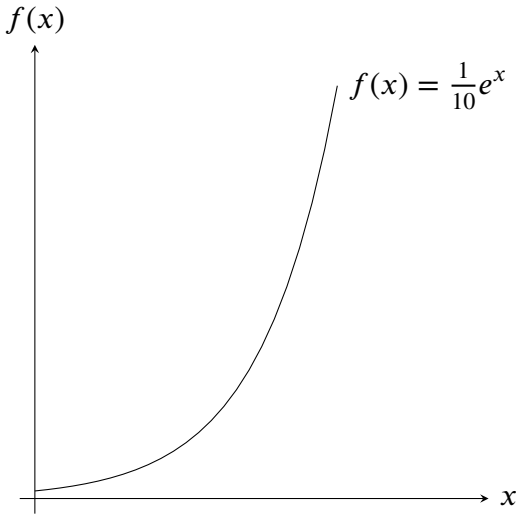
```
\begin{tikzpicture}[sibling distance =80pt]
   \node[cir] {1}
      child {node[cir] {2}}
      child {node[cir] {3}}
      child {node[cir] {4}
         child {node[cir] {5}}
         child {node[cir] {6}}
         child {node[cir] {7}}
      };
\end{tikzpicture}
```



child 关键字用来声明子节点；sibling distance 选项控制相邻节点之间的距离，单位为 pt。

6.5.5 绘制函数图像

```
\begin{tikzpicture}
   \draw[->] (-0.2,0) --(6,0) node[right] {$x$};
   \draw[->] (0,-0.2) --(0,6) node[above] {$f(x)$};
   \draw[domain =0:4] plot (\x ,{0.1* exp(\x)}) node[right] {$f(x)=\frac{1}{10}e^x$};
\end{tikzpicture}
```

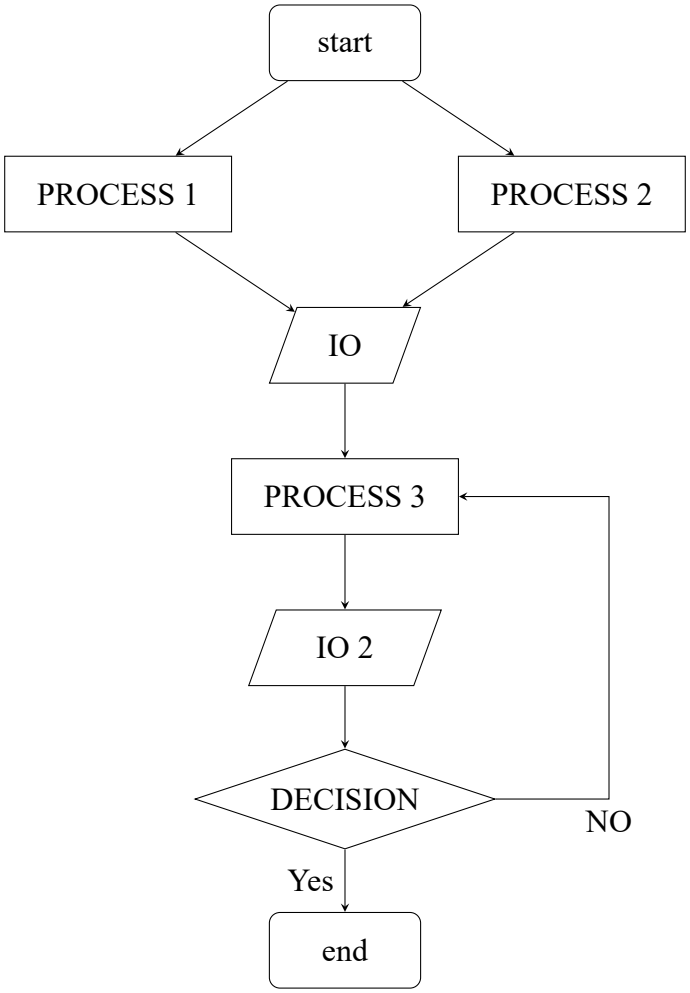


其中 domain 设置了我们想要绘制的范围，起始点和终止点之间用: 隔开。plot 是绘制操作，node 后面 [] 中填写我们文本的位置。

6.6 流程图

```
\usepackage{tikz}
\usetikzlibrary{graphs, positioning, quotes, shapes.geometric,calc}
% 流程图定义基本形状
\tikzstyle{startstop} = [rectangle, rounded corners, minimum width = 2cm, minimum height=1cm,
   text centered, draw = black]
\tikzstyle{io} = [trapezium, trapezium left angle=70, trapezium right angle=110, minimum width
   =2cm, minimum height=1cm, text centered, draw=black]
\tikzstyle{process} = [rectangle, minimum width=3cm, minimum height=1cm, text centered, draw=
   black]
\tikzstyle{decision} = [diamond, aspect = 3, text centered, draw=black]
```

示例：




```
\begin{tikzpicture}[node distance=2cm]
   \node (start) [startstop] {start};
   \node (pro1) [process, below of=start, left of=start, xshift=-1cm] {PROCESS 1};
   \node (pro2) [process, right of=pro1, xshift= 4cm] {PROCESS 2};
   \node (in1) [io, below of=pro1, right of=pro1, xshift=1cm] {IO};
   \node (pro3) [process, below of=in1] {PROCESS 3};
   \node (in2) [io, below of=pro3] {IO 2};
   \node (dec1) [decision, below of=in2] {DECISION};
   \node (end) [startstop, below of=dec1] {end};

   \draw [->](dec1) -- ($(dec1.east) + (1.5,0)$) node[anchor=north] {NO} |- (pro3);
   \graph[use existing nodes=true]{
   start -> pro1 -> in1 -> pro3 -> in2 -> dec1 ->["Yes"left] end ;
   start -> pro2 -> in1 ;
   };
\end{tikzpicture}
```

- 少用、甚至不用具体坐标
  - 第一个 node 省略了坐标，相当于指定了 (0, 0)
  - 从第二个 node 开始，利用 tikz library positioning 提供的选项（例如 below=of <node name>），指定相对位置关系。效果是，每组 node，外框之间的（最小）距离都相同，是 node distance 指定的 2cm；在需要手动调整距离时，可以使用 right=<dimen> of <node name>。
- 简化 arrow 的输入

    - \graph 命令接受(start) -> (step 1) -> (step 2) 这样的输入
    - 使用\graph[use existing nodes=true]，能进一步简化为 start -> step 1 -> step 2
- 简化 node label 的输入
    - 虽然(start) -> (step 1) -> (step 2) 的语法大大简化了输入，但是标记 node label 和绘制非简单 arrow 变麻烦了，带来了学习成本
    - 利用 tiklibraryquotes，将 node label 的输入从 label=[<options>]<text> 简化为”<text>”<options>

6.7 节点文字位置

6.7.1 文字换行

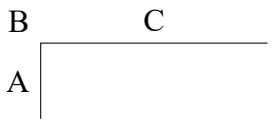
选项中加 align=center，并用\\换行，例如

```
\node at (-0.5,1)[draw, align=center]{example \\ example example};
```

6.7.2 折线文字位置

With -| and |-, midway (or pos=.5) is defined as the point where the two legs meet, regardless of how uneven the two legs are.

```
\coordinate (a) at (0,0);
\coordinate (b) at (3,1);
\draw (a) |- node[pos=.25,left]{A}
   node[pos=.5,above left]{B}
   node[pos=.75,above]{C} (b);
```



6.8 常见需求

6.8.1 描点

\coordinate方法只是创建了点, 但实际上并不会显示 (因为点并没有大小), 而实际中可能需要将相应的点标注出来, 即描点。通过以下方式即可:

```
\node at (O)[circle,fill,inner sep=1pt]{};
```

即在 O 点创建一个 node 对象, 其风格为填充 (fill) 圆 (circle), 宽度为 1pt, 看起来也就是一个可见的点。中为 node 对象的标识, 此处不需要故留白。

或者:

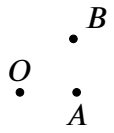
```
\filldraw (O) circle (.05);
```

即在 O 点填充一个半径为 0.05cm 的圆，看起来也是一个可见的点。

6.8.2 指定标签位置

在创建元素时, 可以为其添加标签, 并控制标签出现在元素的相对位置, 如下:

```
\begin{tikzpicture}
   \coordinate[label=above:$O$] (O) at (0,0);
   \coordinate[label=below:$A$] (A) at (0.75,0);
   \coordinate[label=above right:$B$] (B) at (45:1);
\end{tikzpicture}
```

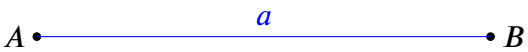


以上代码分别演示将标签放置于元素的上方 | 下方 | 右上方, 共可设置八种方位。

6.8.3 为线添加标签

除了为点标注标签外, 也可为线添加标签，代码如下：

```
\draw[blue] (A) -- (B) node[above,midway](line){$a$};
```

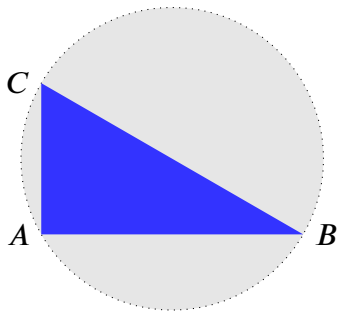


前半部分为画线, 然后创建 node 元素以承载对线条 AB 的标签, 通过 above|midway 控制标签相对于线条 AB 的位置, 即置于线条上方 | 中间。此外, (line) 为该对象设置索引, 方便其他语句引用该对象。(若无需再指向该对象则可以省略)

6.8.4 设定数值包含根式的元素

某些情况下可能需要以根式值作为线段或圆半径设定对象, 而实际上 TikZ 支持在语句中使用 sqrt 函数, 以下为用例:

```
\begin{tikzpicture}[scale=2]
   \coordinate[label=left:$A$] (A) at (0,0);
   \coordinate[label=right:$B$] (B) at ({sqrt(3)},0,0);
   \coordinate[label=left:$C$] (C) at (0,1);
   \draw[fill=gray!20,dotted] ({sqrt(3)/2},0.5) circle (1);
   \fill[blue!80] (A) -- (B) -- (C);
\end{tikzpicture}
```



注意: 务必将 sqrt 运算符用花括号括起来, 以告诉 TikZ 需要运算。

6.8.5 标注直角符号

以下公式可用于创建直角符号:

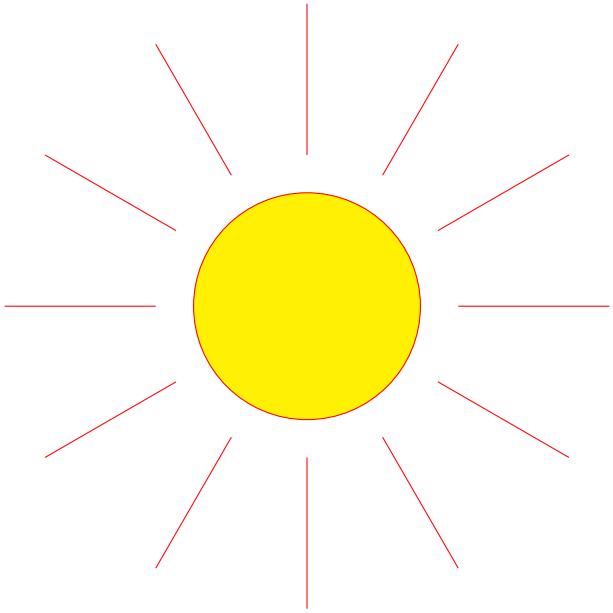
6.8.6 foreach 循环功能

利用foreach命令实现简单的循环功能，其语法格式如下：

```
\foreach \a in {<list>} <commands>
```

其中，foreach表示实现循环功能的命令；\a表示变量；in 表示在什么内，这里是指变量\a只要在 <list> 内；<commands> 表示循环执行的命令。

```
\begin{center}
   \begin{tikzpicture}
      \draw[fill=yellow,draw=red] (0,0) circle [radius=1.5];
      \foreach \x in {0,30,60,90,120,150,180,210,240,270,300,330}
      \draw[red] (\x:2) -- (\x:4);
   \end{tikzpicture}
\end{center}
```

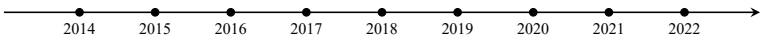


只要 x 在 0,30,60,90,120,150,180,210,240,270,300,330 之中，就绘制直线。直线是极坐标系下的直线，起点坐标的角度为\x，半径为 2；终点坐标的角度为\x，半径为 4。

6.8.7 利用循环功能绘制时间线

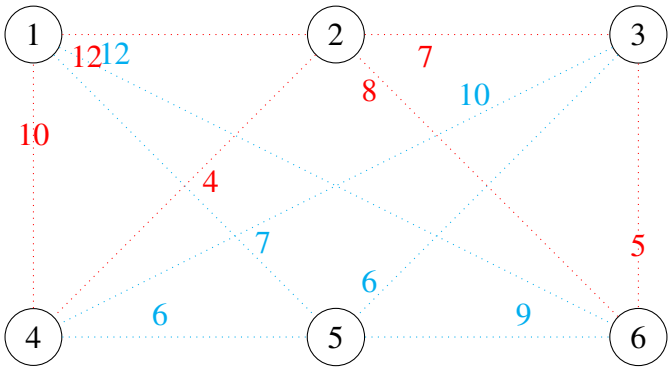
```
\draw [->,semithick] (0,0) -- (10,0);
\foreach \x [count=\i] in {2014,2015,2016,...,2022}
\filldraw ({\i},0) circle (.05) node [below,font=\fontsize{6}{6}\selectfont] {\x};
```

注 如果 j 的取值范围是像自然数一样的, 可以先写前三个, 中间三个点...,, 最后以一个值停止即可, 比如还可以 -1,-0.9,-0.8,⋯,1 得到 21 个值.



6.8.8 多变量同时循环

```
\begin{tikzpicture}[scale=4,auto,swap]
   \foreach \pos/\name in {{(0,0)/4},{(1,0)/5},{(2,0)/6},{(0,1)/1},{(1,1)/2},{(2,1)/3}}
   {\node[draw,circle,black] (\name) at \pos{$\name$};}
   \foreach \source /\dest /\weight /\pos in
   {1/6/12/0.1,3/4/10/0.25,4/5/6/0.40,5/6/9/0.65,1/5/7/0.80,3/5/6/0.95}
   {\path[draw,dotted,cyan] (\source) -- node[pos=\pos, above] {$\weight$} (\dest);}
   \foreach \source /\dest /\weight/\pos in
   {1/2/12/0.1,2/3/7/0.25,2/4/4/0.40,3/6/5/0.65,4/1/10/0.8,6/2/8/0.95}
   {\path[draw,dotted, red] (\source) -- node[pos=\pos, below] {$\weight$} (\dest);}
   \end{tikzpicture}
```



注 当有两个甚至更多的变量伴随时, 使用 `\j/\k/\l` 之类, 取值范围也是对应赋值, 如果某一组 `\j,\k,\l` 的值相等, 可以只提供一个, 即 3/3/3 和一个 3 效果一样.

6.8.9 嵌套循环

```
\foreach \i in {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
{
   \foreach \j in {0, 1, 2, 3, 4}
   {
      ...
   }
}
```

6.8.10 长图片跨页

转自：http://tex.stackexchange.com/questions/22826/breaking-pictures-across-multiple-pages

```latex
\usepackage{adjustbox}
\newsavebox{\mysavebox}
\newlength{\myrest}
\begin{document}

\begin{lrbox}{\mysavebox}
\begin{tikzpicture}
   ...
\end{tikzpicture}
\end{lrbox}
%
\ifdim\ht\mysavebox>\textheight
   \setlength{\myrest}{\ht\mysavebox}
   \loop\ifdim\myrest>\textheight
      \newpage\par\noindent
      \clipbox{0 {\myrest-\textheight} 0 {\ht\mysavebox-\myrest}}{\usebox{\mysavebox}}
      \addtolength{\myrest}{-\textheight}
   \repeat
   \newpage\par\noindent
   \clipbox{0 0 0 {\ht\mysavebox-\myrest}}{\usebox{\mysavebox}}
\else
   \usebox{\mysavebox}
\fi
```

第 7 章 公式

7.1 公式对齐 42
7.1.1 等号对齐 42
7.1.2 大括号连接的多行公式 42

7.1 公式对齐

7.1.1 等号对齐

使用 aligned 环境实现公式的拆分，代码公式如下：

```
\begin{aligned}
   \cos 2x &=\cos^2x - \sin^2x \\
   &=2\cos^2x-1
\end{aligned}
```

$$\begin{aligned}\cos 2x &= \cos^2 x - \sin^2 x \\ &= 2\cos^2 x - 1\end{aligned}$$

7.1.2 大括号连接的多行公式

使用 cases 环境实现公式的组合，& 分隔公式和条件，具体 LaTex 代码如下：

```
D(x) = \begin{cases}
\lim \limits_{x \to 0} \frac{a^x}{b+c},&x<3 \\
\pi,&x=3\\
\int_a^{3b}x_{ij}+e^2dx,&x>3\\
\end{cases}
```

$$D(x) = \begin{cases}\lim\limits_{x\to 0} \frac{a^x}{b+c}, & x<3 \\ \pi, & x=3 \\ \int_a^{3b} x_{ij}+e^2dx, & x>3\end{cases}$$

第 8 章 代码

第 9 章 幻灯片

第 10 章 自定义命令

第 11 章 参考文献

第 12 章 试卷排版

第 13 章 Linux 篇

13.1 WSL 48
13.1.1 安装 WSL 48
13.1.2 安装 Texlive 48
13.1.3 安装 windows 字体 49
13.1.4 VSCode 配置 49
13.1.5 Gitee 同步 50

13.1 WSL

13.1.1 安装 WSL

管理员模式下打开 PowerShell 或 Windows 命令提示符，然后 (需要翻墙) 运行：

```
wsl --install
```

13.1.2 安装 Texlive

1. 下载安装镜像
   清华源
2. 挂载安装镜像

   ```
   sudo mkdir /mnt/texlive
   sudo mount /mnt/d/texlive2023.iso /mnt/texlive
   ```

3. 启动安装程序

   ```
   sudo /mnt/texlive/install-tl % 输入I并回车，开始执行安装。
   ```

4. 安装善后

   ```
   sudo umount /mnt/texlive % 将安装镜像弹出，注意不是unmount!
   sudo rm -r /mnt/texlive % 删除/mnt/texlive文件夹
   ```

5. 配置环境变量

   ```
   sudo apt-get update % 更新apt
   sudo apt install gedit % 安装gedit
   sudo gedit ~/.bashrc
   % 在.bashrc文件的末尾添加如下代码
   export PATH=/usr/local/texlive/2023/bin/x86_64-linux:$PATH
   export MANPATH=/usr/local/texlive/2023/texmf-dist/doc/man:$PATH
   export INFOPATH=/usr/local/texlive/2023/texmf-dist/doc/info:$PATH
   source ~/.bashrc % 重新加载 .bashrc 文件
   $
   ```

6. 重启设备
7. 查看是否安装成功

```
tex --version
```

13.1.3 安装 windows 字体

1. 将 windows 下进入 C 盘的 C:/windows/Fonts 拷贝到 D:/winfonts
2. 新建目录用于存储字体

```
sudo mkdir /usr/share/fonts/winfonts
```

3. 将 windows 系统下的字体拷贝到刚刚创建的 winfonts 目录下

```
sudo cp -r /mnt/d/winfonts/* /usr/share/fonts/winfonts
```

4. 修改字体文件的权限，使 root 用户以外的用户也可以使用

```
cd /usr/share/fonts/
sudo chmod -R 755 winfonts
```

5. 将 texlive 配置文件复制到系统

```
sudo cp /usr/local/texlive/2023/texmf-var/fonts/conf/texlive-fontconfig.conf /etc/fonts/
    conf.d/09-texlive.conf
```

6. 安装 mkfontscale mkfontdir 和 fc-cache 命令

```
% 使mkfontscale和mkfontdir命令正常运行
sudo apt-get install ttf-mscorefonts-installer
% 使fc-cache命令正常运行
sudo apt-get install fontconfig
```

7. 建立字体缓存

```
mkfontscale
mkfontdir
fc-cache -fv
```

13.1.4 VSCode 配置

- hsnips 文件

```
sudo mkdir -p ~/.config/Code/User/hsnips
sudo cp -r /mnt/c/Users/bz/AppData/Roaming/Code/User/hsnips/* ~/.config/Code/User/hsnips
```

- 模版

```
sudo cp /usr/local/texlive/2023/texmf-var/fonts/conf/texlive-fontconfig.conf /etc/fonts/
    conf.d/09-texlive.conf
sudo mkdir -p ~/texmf/tex/latex
sudo cp -r /mnt/c/Users/bz/texmf/tex/latex* /home/bz/texmf/tex
```

13.1.5 Gitee 同步

1. 设置帐号

```
git config --global user.name "zhangbin"
```

2. 设置邮箱

```
git config --global user.email "12267558+vmz@user.noreply.gitee.com"
```

3. 生成密钥

```
ssh-keygen -t rsa -C "12267558+vmz@user.noreply.gitee.com"
% 三次回车即可生成 ssh key
```

4. 查看生成的公钥

```
cat ~/.ssh/id_rsa.pub
```

5. 复制至 gitee
   打开 gitee 官网，设置->SSH 公钥-> 添加 SSH 公钥。
   标题可以任意设置，生成的公钥`id_rsa.pub`内容复制到 gitee，就可以添加 SSH 公钥了。
6. 测试验证是否连接成功

```
ssh -T git@gitee.com
% 如出现以下欢迎信息，说明连接成功，进一步证明上面的配置正确．否则，请仔细检查上面的步骤．
Hi xxx! You've successfully authenticated, but GITEE.COM does not provide shell access.
```

7. 创建本地仓库

```
sudo mkdir ./workspace
cd workspace
```

8. 赋予目录账户权限

```
sudo chown -R bz /home/bz/workspace/
```

9. 把这个目录变成 Git 可以管理的仓库

```
git init
```

10. 把本地仓库与一个云端 Gitee 仓库关联（如果从已有云端仓库拉取直接进行下一步）

```
git remote add origin https://gitee.com/vmz/latex.git
```

11. 拉取

```
git pull https://gitee.com/vmz/latex.git master
```

12. VSCode 每次同步都要输入用户名密码？
    在 VSCode 的终端输入 `git config --global credential.helper store` 命令
    在弹框中输入账号和密码，此时输入一次，以后再 git push /pull 的时候就不用再输账号和密码了。